OFFICIAL OPERATIONAL STANDARD

Data Quality Scanner V2.5

Deployment Guide — Installation, Configuration, Validation & Operational Setup

Target Version: V2.5 **Validated Platform:** Windows 10/11 (64-bit) **Runtime:** Python 3.11+ (64-bit)

Audience: Deployment Engineers, Data Engineers, IT Administrators

1. Purpose and Scope

This guide defines the authoritative, repeatable engineering procedure for packaging, installing, configuring, validating, maintaining, and supporting **Data Quality Scanner V2.5** on target host machines.

Scope of This Document

- **Covered:** Host preparation, Python virtual environment configuration, offline/online dependency resolution, configuration governance, static asset deployment, SQLite persistence lifecycle, desktop GUI launcher setup, headless CLI script automation, formal smoke testing, version update procedures, backup/restore sequences, emergency rollback, and deployment troubleshooting.
- **Not Covered:** End-user analytical navigation, interpretation of specific business defects, manual data remediation workflows, or authoring custom ETL migration scripts. For operational end-user procedures, consult the companion *Data Quality Scanner V2.5 User Guide*.

> 📘 **Companion Relationship to User Guide**
>
> The **User Guide** is intended for migration specialists, data analysts, and project managers for daily assessment tasks. The **Deployment Guide** is strictly for technical personnel responsible for ensuring the scanner is securely installed, correctly calibrated, and reliably operating on the host environment.

Architectural Boundary Notice

Data Quality Scanner V2.5 is implemented and validated exclusively as an **isolated, local standalone application**. The following architectures are *Not applicable to the current V2.5 deployment model*:

- Containerization frameworks (Docker, Kubernetes, Podman).
- Cloud SaaS, microservices, or distributed multi-tenant deployments.
- Centralized database clusters or remote relational database servers.
- Web applications, REST APIs, or background Windows service daemons.
- Binary installers (.msi or .exe installer packages).

2. Deployment Model

The scanner utilizes a single-node, file-based deployment model. Both available user interfaces—the **Desktop GUI** (`main.py`) and the **Headless Batch CLI** (`batch_runner.py`)—execute on top of an identical analytical engine within an isolated Python virtual environment (`.venv`).

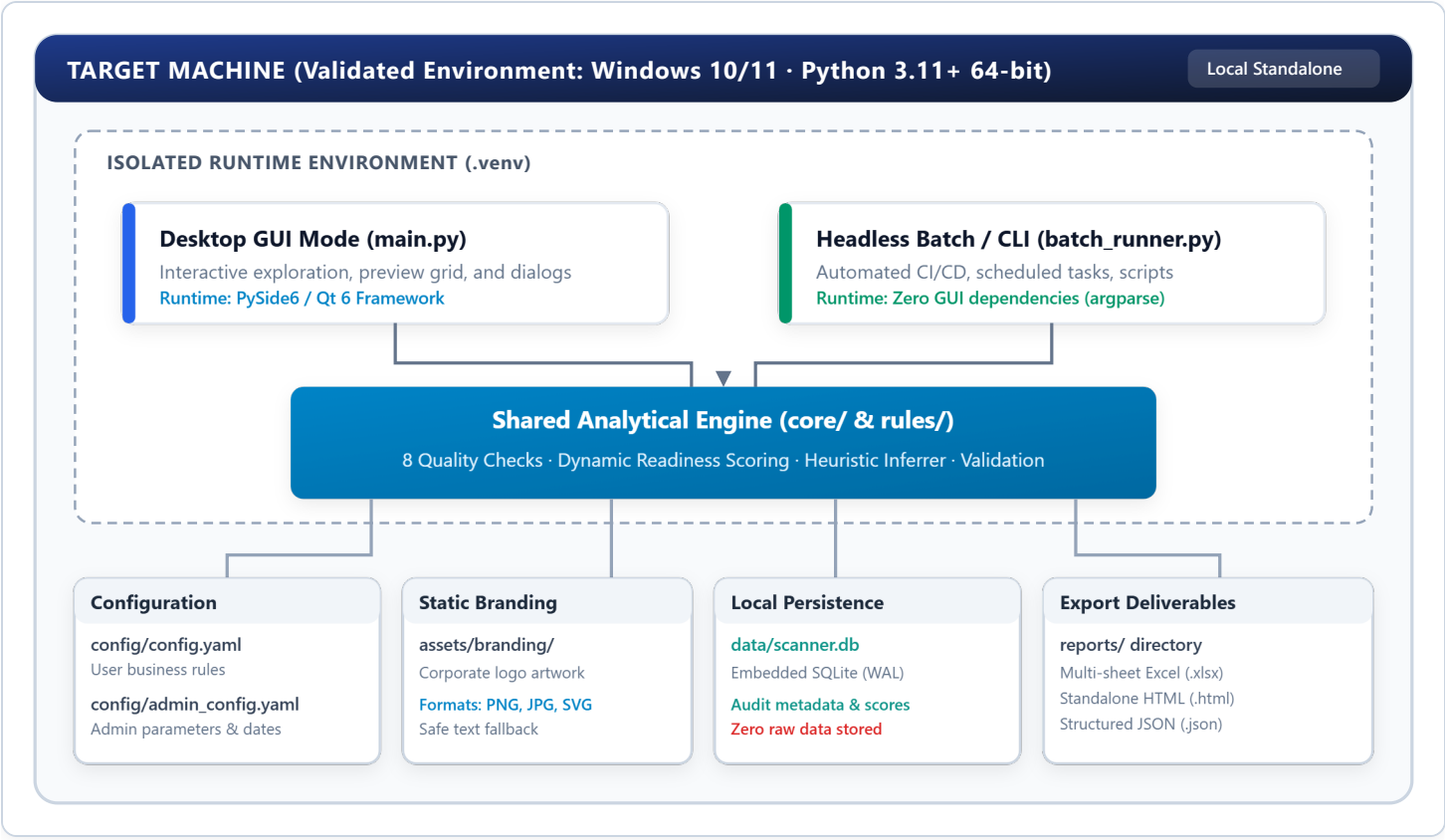


Figure 2.1: Data Quality Scanner V2.5 single-node local deployment model and filesystem boundaries.

Key Architectural Characteristics

Characteristic	Implementation Details
Host Architecture	Single workstation, laptop, or designated virtual machine running Windows 10/11 (64-bit).
Runtime Isolation	Local CPython virtual environment (`.venv`) created via Python's standard `venv` module. Isolates all application dependencies from system packages.
Cloud & Server Independence	Normal V2.5 execution does not require a remote application server or cloud service. Where outbound package installation is restricted, dependencies can be supplied through an approved offline/internal package source.
Persistence Layer	Embedded SQLite database (`data/scanner.db`) managed via WAL mode. Auto-initializes on first execution.
Data Confidentiality	Source records are loaded strictly into host memory for heuristic evaluation. Raw dataset rows are never persisted to the SQLite database.

3. Prerequisites

Before deploying the application, verify that the target host satisfies the technical specifications detailed below.

Category	Specification	Classification	Verification Command
Operating System	Validated Environment: Windows 10 or Windows 11 (64-bit). *(Other platforms supporting the dependencies may function, but should be validated prior to deployment).*	REQUIRED	`[System.Environment]::OSVersion`
Python Engine	Python 3.11 or later (64-bit). Must include `pip` and `venv` modules.	REQUIRED	`python --version`
System Memory	Use a machine with sufficient available memory for the size of the datasets being analyzed. Large datasets may require additional memory because processing is performed in memory.	RECOMMENDED	Task Manager / `systeminfo`
Disk Storage	Sufficient free storage for the application, Python dependencies, SQLite history, and generated reports.	REQUIRED	`Get-PSDrive C`
Filesystem Permissions	Standard user account with Read access to application files, and Write permissions to `data/` (when persistence is enabled) and `reports/` (for outputs).	REQUIRED	Standard folder permissions
PowerShell Execution	PowerShell execution policy may need to permit the local virtual-environment activation script. If activation is blocked, use the documented process-scoped workaround or Windows CMD activation.	DEPLOYMENT NOTE	`Get-ExecutionPolicy`
Display (GUI Only)	GUI requires a supported desktop display; higher resolutions are recommended for comfortable use.	RECOMMENDED	Display Settings

4. Deployment Package Contents

When assembling a production deployment archive (e.g. `migration-dq-scanner-v2.5.zip`), deployment engineers must strictly control which files are packaged.

Path / Artifact	Type	Status	Purpose / Operational Role
`main.py`	Script	REQUIRED	Desktop GUI application entry point (initializes PySide6 / Qt).
`batch_runner.py`	Script	REQUIRED	Headless CLI batch runner (zero GUI dependencies).
`requirements.txt`	Manifest	REQUIRED	Python package dependencies and version constraints specified by `requirements.txt`.
`config/config.yaml`	Configuration	REQUIRED	User business rules (mandatory fields, duplicate keys, relationships).
`config/admin_config.yaml`	Configuration	REQUIRED	Administrator technical engine parameters, date rules, and branding.
`core/`	Package	REQUIRED	Analytical engine (8 quality checks, scoring, ingestion, models).
`gui/`	Package	REQUIRED	Qt visual dashboard components, styles, dialogs, and sections.
`rules/`	Package	REQUIRED	Configuration loading, validation, and admin config management.
`db/`	Package	REQUIRED	SQLite repository, connection manager, and `schema.sql` DDL.
`reports/`	Package	REQUIRED	Report generator modules and HTML Jinja2 template files.
`assets/branding/`	Directory	REQUIRED	Corporate logo artwork referenced by `admin_config.yaml`.
`samples/`	Directory	RECOMMENDED	Standard test datasets used for deployment smoke testing.
`docs/`	Directory	RECOMMENDED	Official User Guide and Deployment Guide documentation.
`requirements-dev.txt`	Manifest	OPTIONAL	Pytest development libraries (only if running automated test suites).
`tests/`	Directory	OPTIONAL	Automated test suite (recommended for staging validation).
`data/scanner.db`	Database	RUNTIME	Local SQLite database. Auto-created at runtime. **Do not package developer's local DB** unless transferring historical records.
`reports/*.xlsx, *.html, *.json`	Output	RUNTIME	Generated output deliverables. **Do not package developer's scan reports**.
`.venv/`	Environment	DO NOT PACKAGE	**Distribution Package:** The `.venv` directory must NOT be included; it must be created fresh on the target host. **Deployed Target:** After installation, the target host contains its own locally created `.venv`.
`__pycache__/`, `.pytest_cache/`	Cache	DO NOT PACKAGE	Bytecode and test caches. Must be purged prior to distribution.

5. Target Directory Structure

The following directory tree illustrates a clean, fully deployed installation on the target host filesystem:

```
C:\Applications\migration-dq-scanner\
├── main.py                     # [Application] Desktop GUI launch entry point
├── batch_runner.py             # [Application] Headless Batch / CLI runner
├── requirements.txt            # [Package] Runtime library requirements
├── requirements-dev.txt        # [Package] Optional test suite dependencies
│
├── config\                     # [Configuration] Application configuration files
│   ├── config.yaml             # User business rules (mandatory fields, duplicate keys)
│   └── admin_config.yaml       # Admin engine parameters, date rules, branding
│
├── core\                       # [Application Code] Core analytical engine
│   ├── checks\                 # The 8 quality check implementation modules
│   ├── inference\              # Deterministic heuristic rule inference
│   ├── ingestion\              # File readers (CSV, Excel xlsx/xls)
│   ├── scoring\                # Readiness scoring and dimension health logic
│   ├── models.py               # Immutable domain data models
│   └── service.py              # Orchestration analysis service
│
├── gui\                        # [Application Code] PySide6 Qt GUI components
│   ├── sections\               # Dashboard UI sections (header, upload, results)
│   ├── style.py                # Corporate Qt dark/light color themes
│   └── main_window.py          # Main application window controller
│
├── rules\                      # [Application Code] Configuration managers
│   ├── config_loader.py        # YAML loader and normalization routines
│   └── admin_config_manager.py # Safe admin config updater and logo handler
│
├── db\                         # [Application Code] Persistence layer
│   ├── connection.py           # SQLite connection factory (WAL mode)
│   ├── repository.py           # ScanRepository data access operations
│   └── schema.sql              # Relational schema DDL definition
│
├── reports\                    # [Code & Output] Multi-format report generators
│   ├── excel_reporter.py       # OpenPyXL multi-sheet Excel generator
│   ├── html_reporter.py        # Standalone HTML report generator
│   ├── json_reporter.py        # Structured JSON generator
│   ├── summary_builder.py      # Statistical findings aggregator
│   └── templates\              # Jinja2 HTML report templates
│
├── assets\                     # [Static Assets] Branding artwork
│   └── branding\               # Company logos (e.g. company_logo.png)
│
├── data\                       # [Runtime Storage] Local SQLite database
│   └── scanner.db              # Embedded scan-history database (auto-created)
│
├── samples\                    # [Validation] Pre-packaged test datasets
│   ├── customers_clean_100pct.csv
│   ├── customers_faulty.csv
│   └── orders_related_to_customers.csv
│
└── .venv\                      # [Isolated Runtime] Python virtual environment (created on target)
    ├── Scripts\                # Python executables and activate scripts
    └── Lib\site-packages\      # Installed third-party packages (PySide6, pandas, etc.)
```

6. Environment Setup

Execute the following numbered procedure on the target host using **Windows PowerShell** (or Command Prompt) to establish the isolated Python environment.

Step 1 — Unpack / Copy the Deployment Package

Extract the deployment package archive into the designated application folder on the target host (e.g. `C:\Applications\migration-dq-scanner`).

Step 2 — Open PowerShell & Navigate to Project Root

Open a PowerShell session and move into the application directory:

```
cd C:\Applications\migration-dq-scanner
```

Step 3 — Verify Python Installation

Confirm that CPython 3.11+ is installed and accessible in the system PATH:

```
python --version
```

Expected output: `Python 3.11.x` (or `3.12.x` / `3.13.x`). Verify it is a 64-bit architecture.

Step 4 — Create Isolated Virtual Environment

Create the local virtual environment inside the application root directory:

```
python -m venv .venv
```

This generates the `.venv` folder containing isolated copies of the Python binaries and standard library.

Step 5 — Configure Execution Policy & Activate Environment

If PowerShell script execution is restricted on the host, permit script execution for the current process, then activate:

```
# Temporarily permit script execution for current PowerShell session
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned -Force

# Activate the virtual environment
.\.venv\Scripts\Activate.ps1
```

For Windows Command Prompt (CMD), use:

```
.venv\Scripts\activate.bat
```

Verification: The shell prompt must immediately display the `(.venv)` prefix.

7. Dependency Installation

With the `(.venv)` environment activated, install the versions and dependency constraints specified by `requirements.txt`.

Online Installation (Host with Internet Access)

```
# Ensure pip is up to date
python -m pip install --upgrade pip

# Install project runtime dependencies
python -m pip install -r requirements.txt
```

Offline / Internal Package Installation

Where outbound package installation is restricted, dependencies can be supplied through an approved offline/internal package source:

1. On an internet-connected staging machine with matching OS and Python version, pre-download all wheels:

   ```
   pip download -r requirements.txt -d .\wheelhouse
   ```

2. Copy the `wheelhouse\` directory onto the target machine inside the project root.
3. Install offline using the local wheel repository:

   ```
   python -m pip install --no-index --find-links=.\wheelhouse -r requirements.txt
   ```

Required Runtime Packages Reference

Package	Constraint in requirements.txt	Application Function
`PySide6`	>=6.6.0	Qt 6 GUI framework (widgets, styling, graphics gauge, layout engine).
`pandas`	>=2.1.0	In-memory data structures, profiling, vector operations, and deduplication.
`openpyxl`	>=3.1.2	Excel workbook ingestion (.xlsx) and multi-sheet audit report creation.
`PyYAML`	>=6.0.1	Parsing and validation of `config.yaml` and `admin_config.yaml`.
`Jinja2`	>=3.1.2	HTML report templating and dynamic artifact generation.
`matplotlib`	>=3.8.0	Chart rendering and score visualization utilities.
`charset-normalizer`	>=3.3.0	Automatic character encoding detection (UTF-8, Windows-1252, ISO-8859) for CSVs.

Dependency Verification

Verify that all packages are cleanly importable in Python:

```
python -c "import PySide6, pandas, openpyxl, yaml, jinja2, matplotlib, charset_normalizer; \
print('SUCCESS: All runtime dependencies imported cleanly.')"
```

8. Application Configuration

Data Quality Scanner V2.5 enforces a strict, two-tier configuration governance model:

> 🔒 **Configuration Governance Rule**
> **User Business Rules** belong strictly in `config/config.yaml`.
> **Administrator Engine Parameters** belong strictly in `config/admin_config.yaml`.
> **Crucial Syntax Notice:** `date_rules` (accepted date formats and column keyword hints) resides in `config/admin_config.yaml`, NOT in `config/config.yaml`.

1. User Business Rules (`config/config.yaml`)

Reviewed and modified by migration specialists and data analysts. Deployment engineers should ensure this file contains default templates for the target project:

- `datasets:` Defines profiles (e.g. `Customer_Master`, `Orders`) specifying `mandatory_fields:` and `duplicate_keys:`.
- `field_character_rules:` Maps column names to standard character regex presets (`alpha_numeric_only`, `email_format`, `numeric_only`, etc.).
- `relationships:` Defines explicit foreign-key linkages (`parent_dataset`, `parent_key`, `child_dataset`, `child_key`).

2. Administrator Parameters (`config/admin_config.yaml`)

Controlled exclusively by deployment engineers and application administrators:

Parameter Group	Default Values	Deployment Impact / Guidance
`checks.enabled`	All 8 checks active	Master toggle list. Do not disable checks unless explicitly requested by the migration project lead.
`scoring.severity_weights`	Critical: 15.0 High: 8.0 Medium: 3.0 Low: 1.0	Penalty points deducted per defect category. Controls readiness sensitivity.
`scoring.readiness_thresholds`	Ready: 90 Minor Cleansing: 75 Needs Cleansing: 50	Threshold bands defining the overall readiness status badge.
`duplicate_rules.thresholds`	High: 5.0% Medium: 1.0%	Percentage of duplicate records triggering High vs Medium severity.
`null_rules.thresholds`	High: 20.0% Medium: 5.0%	Percentage of missing values in a column triggering High vs Medium severity.
`date_rules.accepted_formats`	8 standard date formats	List of strptime patterns (e.g. `%Y-%m-%d`, `%m/%d/%Y`). Add legacy source formats here.
`date_rules.column_hints`	11 column substrings	Keywords used to detect date columns (e.g. `date`, `_dt`, `_at`, `created`).
`row_count_warning_threshold`	500,000	Triggers memory and performance warning callouts for massive datasets.
`branding`	Title, Subtitle, Logo	Header title, descriptor, and corporate logo file path.

9. Branding and Static Assets

The application provides clean, corporate header branding in both the GUI and exported HTML/Excel reports.

Branding Configuration Schema

Configured in `config/admin_config.yaml` under the `branding:` key:

```
branding:
  title: "Data Quality Scanner V2.5"
  description: "Pre-Migration Data Quality Assessment"
  logo:
    enabled: true
    file: "assets/branding/company_logo.png"
```

Deployment Procedures for Corporate Assets

1. Copy the organization's official logo image into `assets/branding/` (e.g. `assets/branding/acme_logo.png`).
2. Supported image formats: `.png`, `.jpg`, `.jpeg`, `.svg`.
3. Update `config/admin_config.yaml` to reference the relative path:

   ```
   file: "assets/branding/acme_logo.png"
   ```

Sizing Architecture & Fallback Behavior

- **Fixed-Height Anchor:** The application header bar maintains a fixed, stable height (64 px). The logo is anchored to the far right.
- **Aspect-Ratio Scaling:** Images are dynamically scaled using strict `KeepAspectRatio`. A large image will never expand the header height or distort text.
- **Graceful Fallback:** If the specified logo file is missing, corrupt, or has `enabled: false`, the application automatically displays clean text branding without throwing errors or crashing.

10. Database and Persistence Setup

Data Quality Scanner V2.5 includes an embedded SQLite database engine to maintain a historical log of completed scans.

Default Database Location & Initialization

- **Default Path:** `data/scanner.db` (relative to the project root).
- **Auto-Initialization:** When the application performs a scan, `db/connection.py` checks if `data/scanner.db` exists. If missing, it automatically creates the `data/` folder and executes `db/schema.sql` to create all required tables, indexes, and relationships.
- **Zero Manual DDL:** Deployment engineers do *not* need to execute SQL scripts or install SQLite binaries; SQLite is embedded directly in Python's standard library.

Operational Database Settings

Setting	Value	Operational Purpose
Journal Mode	`WAL` (Write-Ahead Logging)	Provides crash resilience, concurrency, and high performance for concurrent reads.
Foreign Keys	`PRAGMA foreign_keys = ON;`	Enforces referential integrity across scan records and defect findings.
Row Factory	`sqlite3.Row`	Permits efficient dictionary-style column access during UI reloads.

Privacy Guarantee

> **Data Privacy & Compliance Statement**
>
> The SQLite database stores execution metadata (timestamps, dataset names, file paths, row counts), overall scores, dimension metrics, issue summary counts, and specific defect coordinates (row numbers, column names, error types). **Raw dataset records and proprietary data rows are NEVER saved into the database.**

Disabling Persistence in Batch / CLI Mode

On automated pipelines where scan results must remain strictly ephemeral, persistence can be bypassed by passing either `--no-db` or `--skip-db`:

```
python batch_runner.py samples/customers_clean_100pct.csv --no-db --excel
```

11. GUI Deployment & Launch

Follow this procedure to launch and verify the Desktop GUI interface on the target host.

Manual Launch via Terminal

```
# 1. Navigate to deployment root
cd C:\Applications\migration-dq-scanner

# 2. Activate virtual environment
.\.venv\Scripts\Activate.ps1

# 3. Launch application
python main.py
```

Creating a One-Click Desktop Launcher (Windows Shortcut)

To allow business users to launch the GUI without interacting with PowerShell or virtual environment commands, create a batch launcher file named `run_scanner.bat` in the application root:

```
@echo off
title Data Quality Scanner V2.5
cd /d "%~dp0"
call .venv\Scripts\activate.bat
start "" pythonw main.py
exit
```

Note on `pythonw.exe`: Using `pythonw` launches the GUI window directly without leaving an empty console window open in the background. Create a standard Windows shortcut pointing to this `run_scanner.bat` on the user's Desktop.

GUI Operational Verification

- Main window opens centered on screen with title `Data Quality Scanner V2.5`.
- Top header renders the application title, description, and corporate logo.
- Dataset Upload section is active with `➕ Add CSV File` and `➕ Add Excel Workbook` buttons.

12. Batch / CLI Deployment & Execution

The headless batch runner provides automated data quality assessment without initializing Qt, making it ideal for scheduled jobs, headless build servers, and automated pipelines.

Command Syntax

```
python batch_runner.py [options] DATASET [DATASET ...]
```

Command-Line Arguments Reference

Option / Flag	Type	Default	Description
`DATASET...`	Positional	*Required*	One or more CSV or Excel (`.xlsx` / `.xls`) dataset paths.
`--config, -c CONFIG`	String	`config/config.yaml`	Custom business rules configuration file path.
`--profile, -p PROFILE`	String	None	Dataset profile name defined in configuration (e.g. `Customer_Master`).
`--output, -o OUTPUT`	String	`reports`	Target directory for generated assessment reports.
`--db DB`	String	`data/scanner.db`	Custom SQLite database path for scan persistence.
`--no-db, --skip-db`	Flag	False	Disable persisting scan findings to SQLite.
`--sheet SHEET`	String / Int	`0`	Worksheet name or 0-based index when analyzing Excel workbooks.
`--html`	Flag	False	Generate standalone HTML assessment report.
`--excel`	Flag	False	Generate multi-sheet Excel (`.xlsx`) assessment report.
`--json`	Flag	False	Generate machine-readable JSON structured export.
`--no-reports`	Flag	False	Disable file-based report export entirely.
`--quiet, -q`	Flag	False	Suppress human-readable console progress and summary output.
`--version, -v`	Flag	—	Display program version and exit.
`-h, --help`	Flag	—	Display command-line help documentation and exit.

Process Exit Codes

Exit Code	Status	Technical Meaning
`0`	**Success**	Scan executed and completed cleanly (regardless of data defect severity).
`1`	**CLI Argument Error**	Invalid command-line syntax, unknown flag, or missing dataset parameter.
`2`	**Data / Config Error**	Input file not found, unreadable file format, or invalid YAML syntax.
`3`	**Runtime Error**	Unhandled runtime or engine exception encountered during execution.

13. Report Output Setup

Generated assessment reports provide stakeholder documentation and record-level remediation logs.

Output Destination Directory

- **Default Location:** `reports/` (inside application root).
- **Custom Location:** Specified via the `--output <path>` CLI flag or the GUI Browse button.
- **Directory Auto-Creation:** The application automatically creates the destination directory if it does not exist.

Report Types & Internal Structure

Format	File Extension	Content & Operational Purpose
Excel Workbook	`.xlsx`	Comprehensive multi-sheet audit file (up to 7 dedicated sheets). Includes `Issue Details` with native `A1:H{last_row}` AutoFilter applied to row-level defects, plus independent `Quality Dimensions` diagnostic scores and conditional `Relationships` details.
Standalone HTML	`.html`	Self-contained interactive report with embedded CSS, SVG meters, and collapsible findings tables. Opens in any modern browser without network access.
Structured JSON	`.json`	Machine-readable flat defect arrays and scan metadata for ingestion into downstream automated migration pipelines or data catalog tools.

Maintenance & Archival Guidance

Because large assessment workbooks and JSON exports can accumulate over multiple migration testing cycles, establish an archival routine on the host:

- Periodically compress and archive older files from `reports/` into cold storage.
- Ensure disk monitoring alerts are configured if scanning multi-gigabyte extracts.

14. Deployment Validation & Smoke Test

Before declaring the application ready for production handover, the deployment engineer must execute the complete 15-point deployment test suite detailed below.

Test ID	Validation Area	Action / Command	Expected Result	Pass / Fail
DT-01	Python Runtime	`python --version`	Returns Python 3.11.x or higher (64-bit).	[ ]
DT-02	Environment Isolation	`.\.venv\Scripts\Activate.ps1`	Shell prompt prefix updates to `(.venv)`.	[ ]
DT-03	Dependencies	`python -c "import PySide6, pandas, openpyxl, yaml, jinja2, matplotlib, charset_normalizer"`	Executes with return code `0` and zero import errors.	[ ]
DT-04	Configuration Parsing	`python -c "import yaml; yaml.safe_load(open('config/config.yaml')); yaml.safe_load(open('config/admin_config.yaml'))"`	Both YAML configuration files parse cleanly without syntax errors.	[ ]
DT-05	Branding Assets	`Test-Path assets/branding/company_logo.png`	Returns `True` and file size is non-zero.	[ ]
DT-06	GUI Launch	`python main.py`	Main window opens with header branding and upload buttons visible.	[ ]
DT-07	GUI File Loading	Click `➕ Add CSV File` → select `samples/customers_clean_100pct.csv`	Dataset appears in the loaded-datasets list; the preview displays the first 10 rows and the application shows the dataset's total row and column counts.	[ ]
DT-08	Rule Inference	Click `🚀 Run Data Quality Analysis`	Analysis proceeds successfully; if high-confidence mandatory fields are inferred, the Mandatory Fields Detected dialog is displayed for confirmation.	[ ]
DT-09	Readiness Calculation	Confirm the inferred rules where the dialog appears (or click `Proceed`) and complete the analysis	Readiness score and status are displayed successfully.	[ ]
DT-10	GUI Report Export	Click `📥 Generate & Save Reports`	Files generated in `reports/` (.xlsx, .html, .json); links appear in UI.	[ ]
DT-11	SQLite Persistence	Inspect `data/scanner.db` and UI Scan History	Database file exists; scan row is logged in the Scan History table.	[ ]
DT-12	CLI Interface	`python batch_runner.py --help`	Help message prints usage and argument options; exit code is `0`.	[ ]
DT-13	CLI Batch Execution	`python batch_runner.py samples/customers_clean_100pct.csv samples/orders_related_to_customers.csv --excel --html --no-db`	Console prints summary table, relationship status, and generated report paths.	[ ]
DT-14	CLI Exit Code	`echo $LASTEXITCODE` (PowerShell)	Returns strictly `0`.	[ ]
DT-15	Report Accessibility	Open generated `.xlsx` in Excel and `.html` in browser	Excel displays tabs cleanly with AutoFilter; HTML renders interactive KPI cards.	[ ]

15. Post-Deployment Verification

Upon completing the smoke test suite, execute these final verification checks before formally handing over the system:

Sanitization & Cleanliness Verification

- **Purge Test Reports:** Remove any temporary test reports generated during smoke testing from `reports/` if the client requires a clean initial state.
- **No Hardcoded Workstation Paths:** Confirm that no absolute developer file paths or personal drive letters exist in `config/config.yaml` or `config/admin_config.yaml`. All paths must remain relative to the application root.
- **No Transient Artifacts:** Confirm that no `.pytest_cache/`, `.git/`, or root-level `.tempmediaStorage/` folders exist on the production deployment.

Formal Deployment Sign-Off Record

Sign-Off Criterion	Status	Verified By (Signature / Date)
Environment & Dependencies Validated (DT-01 to DT-03)	[ ] ACCEPTED	_____
Configuration & Static Assets Verified (DT-04, DT-05)	[ ] ACCEPTED	_____
Desktop GUI Functional Test Passed (DT-06 to DT-11)	[ ] ACCEPTED	_____
Headless CLI Functional Test Passed (DT-12 to DT-14)	[ ] ACCEPTED	_____
Deliverable Reports Verified (DT-15)	[ ] ACCEPTED	_____

16. Updating an Existing Deployment

When deploying a minor release, patch, or configuration update to an active installation, follow this safe sequence:

1. **Terminate Running Processes:** Close the Data Quality Scanner GUI and ensure no active user sessions or automated `batch_runner.py` jobs are running.
2. **Create Pre-Update Backup:** Archive the current working state:
   ```
   # Create timestamped backup folder
   mkdir ..\backups\pre-update-backup
   Copy-Item -Recurse config, data, assets, reports ..\backups\pre-update-backup\
   ```
3. **Deploy Updated Application Code:** Overwrite core application packages (`core/`, `gui/`, `rules/`, `db/`, `reports/`, `main.py`, `batch_runner.py`). Do *not* overwrite modified business rules in `config/config.yaml`. The repository's report-generation modules/templates under `reports/` are application code and may be replaced as part of an application update. Generated `.xlsx`, `.html`, and `.json` deliverables must be preserved or archived separately and must not be treated as application source files.
4. **Check Dependency Changes:** Compare the new `requirements.txt` against the existing installation. If dependencies changed:
   ```
   .\.venv\Scripts\Activate.ps1
   python -m pip install -r requirements.txt
   ```
5. **Database Schema Notice:** The application does not contain an automated database migration engine. If an update introduces changes to `db/schema.sql`, existing databases must be patched manually or archived before allowing the application to re-initialize a fresh schema.
6. **Re-Run Smoke Test Suite:** Execute tests `DT-06`, `DT-10`, and `DT-13` to ensure complete operational integrity.

17. Backup / Restore Considerations

Establishing a disciplined backup regime ensures migration audit logs and custom business rules are never lost.

What Must Be Backed Up

Directory / File	Type	Recommended Frequency	Operational Impact if Lost
`config/config.yaml`	Business Configuration	After every rule change	**High:** Loss of custom dataset definitions, mandatory fields, and relationship linkages.
`config/admin_config.yaml`	Admin Configuration	After every parameter change	**High:** Loss of custom severity weights, detection thresholds, and branding settings.
`data/scanner.db`	SQLite Database	Daily during active migrations	**Medium:** Loss of historical scan audit log and dashboard historical reload ability.
`assets/branding/`	Static Assets	After logo changes	**Low:** Loss of corporate artwork (falls back to default header text).
`reports/ (deliverables)`	Deliverable Output	Weekly / Milestone sign-off	**Medium:** Loss of historical generated audit deliverables (.xlsx, .html, .json), which must be archived separately and must not be treated as disposable application files.

Restoration Sequence

To restore a damaged or corrupted installation:

1. Stop all running instances.
2. Copy backed up `config.yaml` and `admin_config.yaml` into `config/`.
3. Copy backed up `scanner.db` into `data/`.
4. Copy backed up logo files into `assets/branding/`.
5. Launch the GUI and verify scan history displays past entries cleanly.

18. Rollback / Redeployment

If an application update causes unforeseen runtime errors or fails smoke testing, perform an immediate emergency rollback:

> ⚠️ **Version Compatibility Warning**
> Never blindly mix configuration files or SQLite databases from disparate major versions. Always restore the exact matching configuration snapshot taken prior to the update.

Emergency Rollback Steps

1. **Terminate Application:** Close the Data Quality Scanner GUI and stop any scanner-specific Batch/CLI process before beginning rollback.
2. **Restore Application Files:** Overwrite application modules with the pre-update release archive.
3. **Restore Database & Configs:** Copy the pre-update `scanner.db` and YAML files from `..\backups\pre-update-backup\` back into their respective directories.
4. **Reactivate Virtual Environment:**

   ```
   .\.venv\Scripts\Activate.ps1
   python -m pip install -r requirements.txt
   ```

5. **Validate Rollback:** Execute smoke test `DT-13` to prove that the restored version executes successfully.

19. Deployment Troubleshooting

Common technical issues encountered during target host setup, along with root causes and corrective actions:

Observed Failure	Probable Root Cause	Corrective Engineering Action
`python : The term 'python' is not recognized`	Python is not installed, or installer option "Add python.exe to PATH" was unchecked.	Reinstall Python 3.11+ (64-bit) ensuring "Add Python to environment variables" is selected.
`Python was not found; run without arguments to install from Microsoft Store`	Windows "App execution aliases" intercepting the python command.	Open *Settings* > *Apps* > *Advanced app settings* > *App execution aliases* and toggle OFF "App Installer" for `python.exe` and `python3.exe`.
`File .venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled`	PowerShell execution policy defaults to `Restricted` on Windows.	Run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned -Force` in PowerShell before activating.
`ModuleNotFoundError: No module named 'PySide6'`	Virtual environment is inactive, or dependencies were not installed into `.venv`.	Ensure prompt shows `(.venv)`, then run `python -m pip install -r requirements.txt`.
`PermissionError: [Errno 13] Permission denied: 'data/scanner.db'`	Target user lacks write permissions in `data/`, or file is locked by an existing process.	Grant full Modify permissions to the local user account on the application folder. Ensure no orphaned python processes have the file open.
`ConfigError: YAML formatting error in config.yaml`	Tab characters or malformed indentation in YAML configuration.	Open YAML file in VS Code or Notepad++; replace all tabs with 2 spaces; validate syntax with `python -c "import yaml; yaml.safe_load(open('config/config.yaml'))"`.
`database is locked (sqlite3.OperationalError)`	Concurrent write access conflict or zombie process holding WAL file lock.	Terminate all Python processes. Check for `scanner.db-wal` and `scanner.db-shm` files; restart the application.
`PermissionError: Permission denied writing to reports/...xlsx`	An existing report with the same filename is currently open in Microsoft Excel.	Close Microsoft Excel on the target machine and retry report generation.
`CLI terminates with exit code 2`	Input dataset path was invalid, file is unreadable, or config syntax is corrupted.	Verify that dataset files exist at the specified relative or absolute paths. Run without `--quiet` to review stderr logs.
`CLI terminates with exit code 3`	Unhandled runtime or engine exception occurred.	Inspect console traceback output; check dataset for corrupted byte sequences or unsupported complex formatting.

20. Security, Permissions & Operational Considerations

Technical operational controls to ensure secure, compliant execution on corporate target workstations:

User & Administrator Permissions Model

Data Quality Scanner V2.5 distinguishes normal user/operator permissions from administrative rights according to the principle of least privilege:

- **Normal Users / Operators:**
  - Read access to application files and configuration required for execution.
  - Write access to `data/` when SQLite persistence is enabled.
  - Write access to `reports/` for report generation.
- **Administrators:**
  - Additional authorized write access to `config/` when changing administrative configuration (e.g., `admin_config.yaml`).
  - Authorized access to `assets/branding/` when updating corporate branding.
- **Execution Privileges:** Normal application execution is designed to run under a standard Windows user account; administrative rights should only be required for host-level installation or enterprise policy changes.

Data Governance & Confidentiality

- **In-Memory Processing:** Source datasets are processed in application memory. Raw dataset rows are not stored in the SQLite scan-history database.
- **Network Independence:** Normal V2.5 execution does not require a remote application server or cloud service. Where outbound package installation is restricted, dependencies can be supplied through an approved offline/internal package source.
- **Report Access Controls:** Generated reports may contain actual field values and should therefore be stored in access-controlled locations according to organizational policy.

21. Deployment Checklist

A comprehensive reference checklist for the deployment engineer to verify prior to formal handover:

Pre-Deployment & Host Preparation

- [ ] Target machine running 64-bit Windows 10/11.
- [ ] Python 3.11+ (64-bit) verified via `python --version`.
- [ ] Windows App Execution Aliases disabled if needed.
- [ ] PowerShell execution policy permits local script activation (or CMD used).
- [ ] Application directory unpacked at clean target path.

Environment & Dependencies

- [ ] Fresh virtual environment created: `python -m venv .venv`.
- [ ] Virtual environment activated: `(.venv)` prompt visible.
- [ ] `pip install -r requirements.txt` executed with exit code 0.
- [ ] Dependency import test verified cleanly.
- [ ] No developer caches (`__pycache__`, `.pytest_cache`) present.

Configuration & Assets

- [ ] `config/config.yaml` reviewed and calibrated.
- [ ] `config/admin_config.yaml` checked (date_rules verified).
- [ ] Corporate logo deployed to `assets/branding/`.
- [ ] Logo path and title verified in `admin_config.yaml`.
- [ ] Zero absolute machine-specific paths present in YAML configs.

Functional Validation & Handover

- [ ] Desktop GUI launched cleanly: `python main.py`.
- [ ] Sample scan performed; readiness score calculated.
- [ ] Excel, HTML, and JSON reports generated and inspected.
- [ ] SQLite persistence verified in `data/scanner.db`.
- [ ] CLI smoke test executed: `batch_runner.py` exit code 0.
- [ ] Desktop launcher (`run_scanner.bat`) created if requested.
- [ ] User Guide & Deployment Guide delivered to client team.

22. Support / Handover Information

Upon concluding the deployment, assemble and transfer the following operational package to the migration project support team:

Handover Deliverables Package

1. **Application Directory:** The target-host deployment directory containing the installed application files, target-host `.venv`, configuration, and required static assets. (Note: The distribution archive provided to the target host does not include the developer's `.venv`; it is created fresh on the target host).
2. **User Guide:** `docs/user-guide/Data_Quality_Scanner_V2.5_User_Guide.pdf` (for business analysts and daily operators).
3. **Deployment Guide:** `docs/deployment-guide/Data_Quality_Scanner_V2.5_Deployment_Guide.pdf` (this operational engineering standard).
4. **Configuration Map:** Documentation of who owns `config/config.yaml` (data analysts) vs. `config/admin_config.yaml` (system administrator).
5. **Smoke Test Results:** Signed copy of the Deployment Validation & Smoke Test results and Post-Deployment Verification checklist.

Operational Maintenance Responsibilities

Role	Primary Responsibilities
Application Administrator	Maintains `config/admin_config.yaml` (scoring weights, date formats, thresholds); manages corporate branding assets; manages database backups and software updates.
Migration Technical Lead	Maintains business rules in `config/config.yaml` (mandatory column definitions, key constraints, table relationships); reviews low readiness scores; approves data sign-offs.
Data Analysts / Operators	Executes daily dataset scans via Desktop GUI or CLI; analyzes defect findings in exported Excel workbooks; performs source data remediation.